2. Develop a C program for processing sparse matrices using an appropriate data structure. The program shall accept suitable sparse matrix data, use an appropriate representation, and perform operations such as addition and transpose. Demonstrate the correctness of the solution using suitable test cases.

Theory:

A sparse matrix is a matrix in which most of the elements are zero. Storing every element wastes memory, so we store only the non-zero elements along with their row and column indices.

An appropriate representation is the triplet representation, in which each non-zero element is stored as:

- Row index
- Column index
- Value

The first row stores the total number of rows, columns, and non-zero elements.

For example, consider the matrix:

$$A=\begin{bmatrix} 0&0&5\\ 0&8&0\\ 3&0&0 \end{bmatrix}$$

Its triplet representation is:

Row	Column	Value
3	3	3
0	2	5
1	1	8
2	0	3

The first row indicates a matrix of size 3×3 with 3 non-zero elements. The remaining rows store the non-zero entries using zero-based indexing.

3. Algorithm

Matrix addition

1. Read the dimensions and non-zero elements of matrices A and B.
2. Check whether their dimensions are equal.
3. Compare their row and column positions.
4. Add elements having identical positions.
5. Copy elements appearing at different positions into the result.
6. Display the resulting sparse matrix.

Matrix transpose

1. Read the sparse matrix.

2. Interchange the row and column indices of every non-zero element.
3. Interchange the number of rows and columns.
4. Display the transposed matrix.

Program:

```
#include <stdio.h>
#define MAX 100

typedef struct {
   int row;
   int col;
   int value;
} Element;

typedef struct {
   int rows;
   int cols;
   int terms;
   Element data[MAX];
} SparseMatrix;

/* Read a dense matrix and store only non-zero elements */
void readSparse(SparseMatrix *s) {
   int value;

   printf("Enter rows and columns: ");
   scanf("%d %d", &s->rows, &s->cols);

   if (s->rows <= 0 || s->cols <= 0 ||
      s->rows * s->cols > MAX) {
      printf("Invalid dimensions. Maximum 100 elements.\n");
      s->rows = s->cols = s->terms = 0;
      return;
   }
```

```
    s->terms = 0;
    printf("Enter matrix elements row-wise:\n");

    for (int i = 0; i < s->rows; i++) {
        for (int j = 0; j < s->cols; j++) {
            if (scanf("%d", &value) != 1) {
                printf("Invalid input.\n");
                s->rows = s->cols = s->terms = 0;
                return;
            }

            if (value != 0) {
                if (s->terms >= MAX) {
                    printf("Too many non-zero elements.\n");
                    s->rows = s->cols = s->terms = 0;
                    return;
                }

                s->data[s->terms].row = i;
                s->data[s->terms].col = j;
                s->data[s->terms].value = value;
                s->terms++;
            }
        }
    }
}

/* Display triplet representation */
void displayTriplet(SparseMatrix s) {
    printf("Row\tColumn\tValue\n");
    printf("%d\t%d\t%d\n", s.rows, s.cols, s.terms);

    for (int i = 0; i < s.terms; i++) {
```

```
        printf("%d\t%d\t%d\n",
            s.data[i].row,
            s.data[i].col,
            s.data[i].value);
    }
}

/* Display the matrix in ordinary matrix form */
void displayDense(SparseMatrix s) {
    int k = 0;

    for (int i = 0; i < s.rows; i++) {
        for (int j = 0; j < s.cols; j++) {
            if (k < s.terms &&
                s.data[k].row == i &&
                s.data[k].col == j) {
                printf("%d\t", s.data[k].value);
                k++;
            } else {
                printf("0\t");
            }
        }
        printf("\n");
    }
}

/* Add two sparse matrices using sorted triplet entries */
SparseMatrix addSparse(SparseMatrix a, SparseMatrix b) {
    SparseMatrix c = {0};

    c.rows = a.rows;
    c.cols = a.cols;

    int i = 0, j = 0;
```

```
  while (i < a.terms && j < b.terms) {
    int posA = a.data[i].row * a.cols + a.data[i].col;
    int posB = b.data[j].row * b.cols + b.data[j].col;

    if (posA < posB) {
      c.data[c.terms++] = a.data[i++];
    } else if (posA > posB) {
      c.data[c.terms++] = b.data[j++];
    } else {
      int sum = a.data[i].value + b.data[j].value;

      if (sum != 0) {
        c.data[c.terms] = a.data[i];
        c.data[c.terms].value = sum;
        c.terms++;
      }

      i++;
      j++;
    }
  }

  while (i < a.terms)
    c.data[c.terms++] = a.data[i++];

  while (j < b.terms)
    c.data[c.terms++] = b.data[j++];

  return c;
}

/* Transpose by interchanging row and column indices */
SparseMatrix transposeSparse(SparseMatrix a) {
```

```
    SparseMatrix t = {0};

    t.rows = a.cols;
    t.cols = a.rows;
    t.terms = a.terms;

    int k = 0;

    /* Visit original columns in increasing order */
    for (int col = 0; col < a.cols; col++) {
        for (int i = 0; i < a.terms; i++) {
            if (a.data[i].col == col) {
                t.data[k].row = a.data[i].col;
                t.data[k].col = a.data[i].row;
                t.data[k].value = a.data[i].value;
                k++;
            }
        }
    }

    return t;
}

int main(void) {
    SparseMatrix a = {0}, b = {0}, sum, trans;

    printf("Enter Matrix A\n");
    readSparse(&a);

    if (a.rows == 0)
        return 1;

    printf("\nEnter Matrix B\n");
    readSparse(&b);
```

```
    if (b.rows == 0)
        return 1;

    printf("\nTriplet representation of A:\n");
    displayTriplet(a);

    printf("\nTriplet representation of B:\n");
    displayTriplet(b);

    if (a.rows == b.rows && a.cols == b.cols) {
        sum = addSparse(a, b);

        printf("\nA + B (Triplet Representation):\n");
        displayTriplet(sum);

        printf("\nA + B (Matrix Form):\n");
        displayDense(sum);
    } else {
        printf("\nAddition not possible: dimensions differ.\n");
    }

    trans = transposeSparse(a);

    printf("\nTranspose of A (Triplet Representation):\n");
    displayTriplet(trans);

    printf("\nTranspose of A (Matrix Form):\n");
    displayDense(trans);

    return 0;
}
```

Output:

Test Case 1: Matrix Addition and Transpose

Test case	Matrix A	Matrix B	Expected result
1. Normal addition	[[1,0],[0,2]]	[[3,0],[0,4]]	[[4,0],[0,6]]
2. Cancellation	[[5,0],[0,2]]	[[-5,0],[0,-2]]	All-zero matrix
3. Zero matrix	[[0,0],[0,0]]	[[1,0],[0,2]]	Same as B
4. Rectangular transpose	[[1,0,2],[0,3,0]]	Not required for transpose	[[1,0],[0,3],[2,0]]
5. Dimension mismatch	2 × 3 matrix	3 × 2 matrix	Addition rejected